New Horizons®

Centro de Capacitación

Curso

Desarrollo Full Stack con C# y Angular

Docente: Edwin Barrientos



New Horizons®
Centro de Capacitación

Sesión 7 — Bisagra M2 / M3:

Cierre de Backend e Introducción a Angular



Objetivo del módulo: blindar la API con FluentValidation, middleware global y ProblemDetails (1h) e iniciar el desarrollo del frontend con Angular CLI, TypeScript y Standalone Components (2h).



New Horizons®
Centro de Capacitación

Agenda

- Validación robusta de peticiones con FluentValidation
- Middleware global de excepciones y estándar ProblemDetails (RFC 7807)
- Colección de pruebas de integración con Postman / .http
- Introducción a Angular 18: Angular CLI y Standalone Components
- TypeScript esencial para Angular: Tipos, interfaces y data binding



New Horizons®
Centro de Capacitación

Sesión 1:

Cierre de Backend e Introducción a Angular

La sesión bisagra: Puente entre dos mundos

New Horizons®
Centro de Capacitación

De la solidez del Backend a la interactividad del Frontend: cerramos el Módulo 2 blindando la API para producción y encendemos el motor de Angular para crear una UI profesional.



Aduana de datos

FluentValidation y ProblemDetails: respuestas de error estandarizadas.



Angular Moderno

Angular CLI 18, Standalone Components y adiós a los viejos NgModules.



Full Stack real

Mismos modelos de datos en C# y TypeScript conversando por HTTP.

La aduana de datos: ¿Por qué FluentValidation?

New Horizons®
Centro de Capacitación

Validación declarativa y fuertemente tipada: separa las reglas de validación de los modelos y controladores, haciendo el código limpio y reusable.

Reglas legibles RuleFor(x => x.Motivo).NotEmpty().MaximumLength(200).

Reglas compuestas Validar colecciones: RuleForEach(x => x.Detalles).SetValidator(...).

Mensajes claros Mensajes personalizados por regla para informar al usuario de Angular.

Validaciones complejas Reglas condicionales con When() y validaciones asíncronas con MustAsync().

- Filtra la petición antes de que toque cualquier servicio de negocio.

Validador de Solicitud con FluentValidation

New Horizons®
Centro de Capacitación

CrearSolicitudValidator.cs

```
public class CrearSolicitudValidator : AbstractValidator<CrearSolicitudDto>
{
    public CrearSolicitudValidator( )
    {
        RuleFor(x => x.Motivo).NotEmpty( ).WithMessage("El motivo es obligatorio.")
            .MaximumLength(200).WithMessage("Máximo 200 caracteres.");
        RuleFor(x => x.CentroCostoId).GreaterThan(0);
        RuleFor(x => x.Detalles).NotEmpty( ).WithMessage("Debe incluir al menos un detalle.");
        RuleForEach(x => x.Detalles).ChildRules(d => {
            d.RuleFor(x => x.Concepto).NotEmpty( );
            d.RuleFor(x => x.Importe).GreaterThan(0);
        });
    }
}
```

Registro en Program.cs

New Horizons®
Centro de Capacitación

ProgramValidation.cs

```
// Registro automático de todos los validadores del ensamblado
builder.Services.AddValidatorsFromAssemblyContaining<CrearSolicitudValidator>( );

// Filtro de validación automático que devuelve 400 Bad Request
builder.Services.Configure<ApiBehaviorOptions>(opt =>
{
    opt.InvalidModelStateResponseFactory = context =>
    {
        var errores = context.ModelState.ToProblemDetails( );
        return new BadRequestObjectResult(errores);
    };
});
```

Middleware global de excepciones

New Horizons®
Centro de Capacitación

GlobalExceptionMiddleware.cs

```
// Captura cualquier error no controlado sin exponer stack traces
public class GlobalExceptionMiddleware : IMiddleware
{
    public async Task InvokeAsync(HttpContext context, RequestDelegate next)
    {
        try { await next(context); }
        catch (ReglaDominioException ex) { await EscribirProblemDetails(context, 400, ex.Message); }
        catch (Exception ex) { await EscribirProblemDetails(context, 500, "Error interno del servidor."); }
    }
}
```

El estándar RFC 7807: ProblemDetails

New Horizons
Centro de Capacitación

Respuestas de error predecibles para el Frontend: ProblemDetails define un formato universal para reportar errores en APIs HTTP modernas.



type / title

URI descriptiva y título corto legible por humanos (ej. 'Bad Request').



status

Código HTTP numérico concordante (400, 401, 403, 404, 500).



errors / detail

Diccionario con el campo exacto que falló y la lista de mensajes.

Pruebas de integración automatizadas

New Horizons
Centro de Capacitación

Confianza antes de construir la UI: probamos cada endpoint, validación y respuesta de error con una suite de pruebas automatizadas.



Archivo .http

Pruebas ejecutables directamente desde VS Code o Visual Studio.



Colección Postman

Tests automáticos con pm.test() para validar código 200, 201 y esquemas.



Entregable M2

El backend queda 100% certificado y listo para ser consumido.

¡Hola Frontend! El ecosistema Angular

New Horizons®
Centro de Capacitación

Framework empresarial por excelencia: mantenido por Google, proporciona arquitectura estricta, TypeScript nativo y estabilidad a largo plazo.



SPA Moderna

Carga una sola vez y actualiza la vista sin recargar el navegador.



Standalone Architecture

Elimina la complejidad de los NgModules: componentes directos y livianos.



Angular CLI

Herramienta oficial para crear, servir, probar y empaquetar la app.

Creación del proyecto Frontend

New Horizons®
Centro de Capacitación

```
terminal
# 1) Instalar Angular CLI de forma global
npm install -g @angular/cli
# 2) Crear el proyecto con arquitectura Standalone
ng new solicitudes-frontend --routing --style=css --ssr=false
# 3) Levantar el servidor de desarrollo
cd solicitudes-frontend
ng serve --open
# La aplicacion queda corriendo en http://localhost:4200
```

Anatomía de un Standalone Component

New Horizons®
Centro de Capacitación

navbar.component.ts

```ts
import { Component } from "@angular/core";
import { CommonModule } from "@angular/common";

@Component({
  selector: "app-navbar",
  standalone: true, // Sin NgModules
  imports: [CommonModule],
  templateUrl: "./navbar.component.html",
  styleUrl: "./navbar.component.css"
})
public class NavbarComponent {
  titulo = "Sistema de Solicitudes de Dinero";
}
```

TypeScript esencial: Tipado y Seguridad

New Horizons®
Centro de Capacitación

solicitud.model.ts

```typescript
// Tipado estático con interfaces que reflejan los DTOs de C#
export type EstadoSolicitud = "Borrador" | "Enviada" | "EnRevision" | "Aprobada" | "Rechazada" | "Pagada";

export interface SolicitudDetalle {
  id: number;
  concepto: string;
  importe: number;
}

export interface Solicitud {
  id: number; motivo: string; montoTotal: number;
  estado: EstadoSolicitud; fechaCreacion: string;
  detalles: SolicitudDetalle[ ];
}
```

Sincronización Backend y Frontend

Un solo lenguaje entre servidores y navegadores: los DTOs en C# y las Interfaces en TypeScript deben ser un espejo exacto para evitar errores en tiempo de ejecución.



C# Backend

public record SolicitudDto(int Id, string Motivo, decimal MontoTotal...);



JSON HTTP

camelCase automático de ASP.NET Core: id, motivo, montoTotal.



TS Frontend

export interface Solicitud { id: number; motivo: string; montoTotal: number; };

Data Binding: Enlace de datos

New Horizons®
Centro de Capacitación

data-binding.html

```
<!-- 1) Interpolación: mostrar texto en HTML -->
<h2>{{ solicitud.motivo }}</h2>

<!-- 2) Property Binding: enlazar atributos con corchetes [ ] -->
<button [disabled]="solicitud.estado !== \'Borrador\'">Editar</button>

<!-- 3) Event Binding: capturar clicks con parentesis ( ) -->
<button (click)="enviarSolicitud(solicitud.id)">Enviar</button>

<!-- 4) Two-Way Binding: sincronización bidireccional [(ngModel)] -->
<input [(ngModel)]="motivoBusqueda" placeholder="Buscar motivo...">
```

Primer componente funcional: Card

solicitud-card.html

```
<!-- solicitud-card.component.html -->
<div class="card-solicitud">
  <div class="header">
    <span class="badge">{{ solicitud.estado }}</span>
    <strong>#{{ solicitud.id }}</strong>
  </div>
  <p class="motivo">{{ solicitud.motivo }}</p>
  <div class="footer">
    <span class="monto">{{ solicitud.montoTotal | currency:\'PEN\':\'symbol\' }}</span>
    <button (click)="verDetalle.emit(solicitud.id)">Ver Detalle</button>
  </div>
</div>
```

Estilos encapsulados en CSS

New Horizons®
Centro de Capacitación

solicitud-card.css

```css
/* Los estilos de un componente no contaminan al resto de la app */
.card-solicitud {
  background: #ffffff; border: 1px solid #e0e0e0;
  border-radius: 8px; padding: 16px; margin-bottom: 12px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.05);
}
.badge {
  background: #dceefb; color: #024b98;
  padding: 4px 8px; border-radius: 4px; font-weight: bold;
}
.monto { font-size: 1.2rem; font-weight: bold; color: #2ea84f; }
```

El flujo completo de la petición

New Horizons
Centro de Capacitación

1 2 3

Usuario en Angular

- Llena el formulario en la UI
- El componente valida en local
- Dispara petición HTTP POST

Aduana ASP.NET Core

- Middleware captura la llamada
- FluentValidation audita datos
- Devuelve ProblemDetails si falla

Persistencia y Retorno

- EF Core guarda en BD
- Devuelve 201 Created con DTO
- Angular actualiza la tarjeta

Checklist de transición Backend-Frontend

New Horizons®
Centro de Capacitación

Cerrando el backend con broche de oro: antes de sumergirse de lleno en la UI de Angular, certifiquemos la entrega de la API.



FluentValidation

Todas las peticiones entrantes validadas con mensajes en español.



ProblemDetails

Manejo global de excepciones sin fugas de stack trace.



Angular 18 Listo

ng serve levantado y primer componente renderizando datos.

Errores comunes en validación y Angular

New Horizons®
Centro de Capacitación

Evita esto

- ✗ Validar datos con IFs anidados en cada controlador
- ✗ Devolver excepciones en texto plano o HTML al frontend
- ✗ Usar módulos @NgModule en proyectos nuevos de Angular 18
- ✗ Usar el tipo 'any' indiscriminadamente en TypeScript
- ✗ Mutar el DOM directamente con document.getElementById

→

Haz esto

- ✓ Centralizar reglas con clases AbstractValidator
- ✓ Estandarizar errores con RFC 7807 ProblemDetails
- ✓ Construir aplicaciones con Standalone Components
- ✓ Declarar interfaces estrictas para cada modelo de datos
- ✓ Dejar que Angular gestione el DOM con Data Binding

Buenas prácticas de desarrollo Full Stack

New Horizons®
Centro de Capacitación

Consistencia en ambos lados del stack: la disciplina en el Backend facilita el trabajo en el Frontend y reduce el tiempo de desarrollo a la mitad.

- ✓ Mantén los nombres de propiedades en camelCase para el JSON de la API.
- ✓ Nunca confíes en los datos del frontend: valida siempre en el servidor.
- ✓ Crea interfaces TypeScript idénticas a los DTOs de C#.
- ✓ Divide la interfaz de Angular en componentes pequeños y reusables.
- ✓ Documenta cada endpoint con Swagger para agilizar la integración.

Resumen de la sesión

New Horizons®
Centro de Capacitación



Validación

FluentValidation como aduana de entrada.



Middleware

ProblemDetails captura excepciones sin fugas.



Postman / .http

Backend certificado con tests automáticos.



Angular 18

Standalone Components y adiós NgModules.



TypeScript

Interfaces que reflejan exactamente los DTOs.



Binding

Interpolación, properties y eventos reactivos.

PREGUNTAS...

New Horizons®
Centro de Capacitación

1. ¿Por qué FluentValidation es superior a validar campos con IFs dentro del controlador?
2. ¿Qué beneficios aporta responder con ProblemDetails (RFC 7807) en lugar de strings sueltos?
3. ¿Qué son los Standalone Components y por qué reemplazan a los tradicionales NgModules?
4. ¿Cómo se sincronizan los tipos de TypeScript con los DTOs de C# en el backend?
5. ¿Cómo se comunican el template HTML y la clase TypeScript mediante Data Binding?

New Horizons
Centro de Capacitación

Conclusiones

- ✔️ FluentValidation actúa como la aduana de la API: ninguna petición corrupta llega al servicio.
- ✔️ El middleware global captura cualquier error no controlado y devuelve un ProblemDetails uniforme.
- ✔️ Angular 18 simplifica radicalmente la arquitectura al usar componentes standalone sin NgModules.
- ✔️ TypeScript nos da tipado estático y autocompletado en el frontend, previniendo errores en tiempo de diseño.
- ✔️ La sesión 7 une ambos mundos: dejamos una API lista para ser consumida y encendemos el frontend.

GRACIAS